

All students should watch the videos and review the readings accompanying this LGA in preparation for this assignment and the next lecture.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

1. For each item, indicate whether it is a constant time operation or not.

- constant / not constant : assignment of a **vector** (not just an element in the vector)
- constant / not constant : accessing an array of **int** at index n
- constant / not constant : calling a function with signature **void fn()**
- constant / not constant : addition (concatenation) of two **string** values
- constant / not constant : assignment of a **float** value
- constant / not constant : squaring an **int**
- constant / not constant : accessing the n^{th} node in a linked list
- constant / not constant : Comparing two integers to see which is larger
- constant / not constant : dereferencing a pointer
- constant / not constant : raising an **int** to power k
- constant / not constant : multiplication of two **double** values
- constant / not constant : printing the integer value n of arbitrary size in base 10

2. Rank the following functions (from 1 to 8) in ascending order of their growth rates.

$$n^{4/3}$$

$$\sqrt{n} \log n$$

$$2^n$$

$$n^2$$

$$1$$

$$\log n$$

$$n!$$

$$n$$

3. Simplify the following expressions in “big-O”:

- $O(n^3 + n + 1)$
- $O(\log n + n \log n - n)$
- $O\left(\frac{2n^2}{3} + 100n + 42\right)$
- $O\left(\sum_{i=1}^n i\right) = O(1 + 2 + 3 + \dots + n)$
- $O(2^n + n^3)$
- $O(n^{100} + n! + \sqrt{n})$

Group questions

Analyze the following functions and give an asymptotic complexity in terms of n . Assume that printing small values and short strings (using `cout`) is a constant time operation. Divide these up so that each function has at least three analyses from different group members.

1. (Basic)

```
void times_table(unsigned int n) {  
    for (unsigned int i = 0; i <= n; i++) {  
        for (unsigned int j = 0; j <= n; j++) {  
            cout << i * j << " ";  
        }  
        cout << endl;  
    }  
}
```

2. (Intermediate)

```
void print_powers(int n) {
    int val = 1;
    int p = 0;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            cout << n << "^" << p << " = " << val << "\t";
            val = val * n;
            p++;
        }
        cout << endl;
    }
}
```

3. (Intermediate)

```
void aesop(unsigned int n) {
    unsigned tortoise = 0;
    unsigned hare = 0;
    while (tortoise < n && hare < n) {
        if (tortoise > hare) {
            hare += (n - tortoise) / 2;
        }
        tortoise++;
    }
    if (tortoise > hare) cout << "Tortoise wins!";
    else if (hare > tortoise) cout << "Hare wins!";
    else cout << "It's a tie!";
}
```

For Further Challenge

Bubblesort is a famous ([infamous?](#)) sorting algorithm mainly used in the teaching of algorithms. Its pseudo-code can be found below (note: this is different than the algorithm expressed in your textbook).

```
function BUBBLESORT( $A[1..n]$ )    //  $A$  is an array of  $n$  comparable values
    repeat
         $swapped \leftarrow \text{false}$ 
        for  $i = 2$  to  $n$  do
            if  $A[i - 1] > A[i]$  then
                SWAP( $A[i - 1]$ ,  $A[i]$ )
                 $swapped \leftarrow \text{true}$ 
    until not  $swapped$ 
```

This algorithm performs better on some inputs than others. Can you describe a *best case* input, on which the algorithm performs the fewest comparisons? What is the complexity of BUBBLESORT in the best case? Next, can you describe a *worst case* input, and analyze the algorithm in the worst case?